

PROPOSAL TUGAS IMPLEMENTASI KRIPTOGRAFI



CARAKA Desktop: Perancangan dan Implementasi Protokol Komunikasi *Mesh Offline* Terdesentralisasi Berbasis *Lightweight Cryptography* (Ascon NIST SP 800-232)

**Andika Aryansyach Fauzan (2322101878)
Mahendra Nur Hidayat (2322101937)
Rafi Putra Fadlurrahman (2322101963)**

**Rekayasa Sistem Kriptografi
Politeknik Siber dan Sandi Negara
2026**

DAFTAR ISI

	Halaman
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL.....	iv
BAB I PENDAHULUAN.....	1
I.1 LATAR BELAKANG.....	1
BAB II RUMUSAN MASALAH.....	5
II.1 RUMUSAN MASALAH	5
BAB III DESAIN ARSITEKTUR.....	5
III.1 OBJEK PENELITIAN	5
III.2 ANALISIS KEBUTUHAN	6
III.2.1 Kebutuhan Fungsional	8
III.2.2 Kebutuhan Non-Fungsional	8
III.2.3 Kebutuhan Perangkat Keras	9
III.2.4 Kebutuhan Perangkat Lunak	10
III.3 DESAIN RANCANGAN	11
III.3.1 Arsitektur Sistem.....	11
III.3.2 Desain Protokol CLAMP	12
III.3.3 Desain Mekanisme Sinkronisasi	12
III.3.4 Desain Evaluasi Penelitian	13
DAFTAR PUSTAKA	14

DAFTAR GAMBAR

	Halaman
Gambar 3.1 Arsitektur CARAKA.....	7
Gambar 3.2 Arsitektur Keseluruhan Sistem.....	11
Gambar 3.3 Antarmuka Caraka.....	11
Gambar 3.4 Skema Alur Kerja Pengiriman Pesan <i>End-to-End</i>	12

DAFTAR TABEL

	Halaman
Tabel 3.1 <i>Tools</i> dan persyaratan instalasi	6
Tabel 3.2 Karakteristik Perangkat Laptop	7
Tabel 3.3 Kebutuhan Non-Fungsional Sistem CARAKA Desktop	8
Tabel 3.4 Spesifikasi Minimum Perangkat Keras	9
Tabel 3.5 <i>Stack</i> Teknologi Perangkat Lunak CARAKA Desktop	10
Tabel 3.6 Depedensi Perangkat Pengembangan	10
Tabel 3.7 Parameter Evaluasi Microbenchmark Kriptografi.	13
Tabel 3.8 Parameter Evaluasi Network-Level Benchmark	13

BAB I

PENDAHULUAN

I.1 LATAR BELAKANG

Perkembangan teknologi komunikasi digital telah mengubah cara manusia bertukar informasi secara cepat dan *real-time*. Berbagai aplikasi pesan modern seperti WhatsApp, Telegram, Signal, dan *platform* komunikasi lainnya bergantung pada infrastruktur internet yang terpusat melalui server, pusat data, penyedia layanan internet, serta jaringan *backbone* global. Meskipun pendekatan ini menawarkan kemudahan dan skalabilitas tinggi, ketergantungan terhadap infrastruktur terpusat menciptakan risiko *single point of failure* yang dapat menyebabkan terhentinya komunikasi ketika terjadi gangguan jaringan, kegagalan server, bencana alam, maupun pembatasan akses jaringan pada kondisi tertentu [1].

Dalam berbagai situasi darurat seperti gempa bumi, banjir, kebakaran hutan, konflik sosial, maupun pemadaman listrik berskala besar, infrastruktur komunikasi sering kali menjadi salah satu komponen yang paling terdampak. Akibatnya, masyarakat dan petugas lapangan mengalami kesulitan melakukan koordinasi ketika komunikasi justru menjadi kebutuhan yang paling penting. Selain masalah ketersediaan layanan (*availability*), model komunikasi terpusat juga menimbulkan persoalan privasi karena metadata komunikasi pengguna tetap dapat diamati oleh penyedia layanan maupun pihak ketiga yang memiliki akses terhadap infrastruktur jaringan [2].

Sebagai alternatif terhadap model komunikasi terpusat, jaringan *Peer-to-Peer* (P2P) berbasis *mesh networking* menawarkan pendekatan yang lebih resilien dan terdesentralisasi. Pada jaringan *mesh*, setiap perangkat dapat berfungsi sebagai pengirim, penerima, sekaligus *node* perantara (*relay*) yang membantu meneruskan paket ke perangkat lain. Dengan mekanisme ini, komunikasi dapat berlangsung tanpa ketergantungan terhadap server pusat maupun konektivitas internet. Pesan dapat dikirim melalui beberapa jalur alternatif (*multi-hop routing*) hingga mencapai tujuan akhir. Karakteristik tersebut menjadikan jaringan *mesh* sangat relevan untuk komunikasi darurat, komunitas lokal, wilayah terpencil yang minim infrastruktur telekomunikasi, maupun lingkungan yang membutuhkan privasi komunikasi tinggi [3].

Meskipun menawarkan ketahanan yang lebih baik dibandingkan arsitektur terpusat, penerapan keamanan pada jaringan *mesh* menghadirkan tantangan yang lebih kompleks. Setiap *node* perantara harus dapat meneruskan paket tanpa memiliki kemampuan untuk membaca isi pesan. Selain itu, sistem juga harus mampu menjamin kerahasiaan data, integritas pesan, autentikasi pengirim, perlindungan

terhadap serangan *replay*, serta menjaga privasi metadata *routing*. Tantangan tersebut menjadi semakin sulit ketika komunikasi dilakukan pada lingkungan *offline-first* dan *delay-tolerant network* yang tidak dapat menjamin urutan maupun waktu kedatangan paket [4].

Berbagai penelitian dan implementasi sistem komunikasi *mesh* telah dikembangkan untuk mengatasi permasalahan tersebut. Salah satu implementasi yang populer adalah Meshtastic, yaitu *platform* komunikasi *mesh* berbasis LoRa yang berjalan pada perangkat ESP32 dan NRF52. Meshtastic menggunakan AES-256 untuk enkripsi kanal komunikasi dan mendukung pertukaran kunci berbasis X25519. Namun demikian, sistem ini masih mengirimkan informasi *routing* dalam bentuk *plaintext* sehingga topologi jaringan dapat dianalisis melalui pengamatan lalu lintas jaringan. Selain itu, penggunaan tanda tangan digital Ed25519 menghasilkan *overhead* sebesar 64 byte pada setiap paket yang secara signifikan mengurangi kapasitas *payload* pada kanal ber-*bandwidth* rendah seperti LoRa [5].

Pendekatan lain diterapkan oleh Berty Protocol yang dibangun di atas IPFS dan OrbitDB dengan konsep *offline-first communication*. Sistem ini memanfaatkan Ed25519 dan X25519 untuk autentikasi dan pertukaran kunci. Meskipun mendukung komunikasi terdesentralisasi, mekanisme sinkronisasi berbasis *gossip protocol* menghasilkan *overhead* yang relatif tinggi sehingga kurang optimal untuk lingkungan dengan keterbatasan *bandwidth* [6].

Briar merupakan aplikasi komunikasi yang dirancang untuk aktivis, jurnalis, dan pengguna yang membutuhkan privasi tinggi. Briar mengimplementasikan konsep yang serupa dengan *Signal Protocol* menggunakan mekanisme *Double Ratchet* dan *Perfect Forward Secrecy*. Namun, proses *routing* pesan pada Briar bergantung pada hubungan sosial antar pengguna (*social graph*), sehingga tidak cocok untuk jaringan *mesh* generik yang bersifat dinamis dan terbentuk secara spontan [7].

Sementara itu, Secure Scuttlebutt (SSB) menggunakan pendekatan *append-only signed log* yang memungkinkan sinkronisasi data secara terdesentralisasi. Setiap *entri log* ditandatangani menggunakan Ed25519 dan membentuk rantai hash yang tidak dapat dimodifikasi. Akan tetapi, mekanisme ini menciptakan rekam jejak komunikasi yang bersifat permanen sehingga kurang sesuai untuk kebutuhan komunikasi privat yang bersifat sementara (*ephemeral messaging*) [8].

Berdasarkan analisis terhadap sistem-sistem tersebut, terdapat beberapa celah penelitian (*research gap*) yang masih belum teratasi secara optimal. Pertama, sebagian besar sistem masih menggunakan mekanisme autentikasi berbasis tanda tangan digital yang menghasilkan *overhead* paket relatif besar. Kedua, metadata *routing* masih banyak diekspos dalam bentuk yang dapat dianalisis oleh pihak

ketiga. Ketiga, mekanisme *forward secrecy* yang umum digunakan memerlukan asumsi konektivitas dan sinkronisasi yang sulit dipenuhi pada jaringan *delay-tolerant*. Keempat, sebagian besar sistem belum dirancang secara khusus untuk mengintegrasikan keamanan, efisiensi *bandwidth*, dan operasi *offline-first* dalam satu arsitektur yang ringan.

Untuk menjawab permasalahan tersebut, penelitian ini mengusulkan pengembangan CARAKA Desktop (*Cryptographically Authenticated Relay Architecture for Knowledge and Autonomy*), yaitu aplikasi komunikasi desktop berbasis jaringan *mesh* terdesentralisasi yang dirancang khusus untuk beroperasi pada lingkungan *offline-first*. Sistem ini mengimplementasikan protokol komunikasi yang dinamakan *Compact Lightweight Authenticated Mesh Protocol* (CLAMP) untuk menyediakan komunikasi yang aman, efisien, dan tidak bergantung pada infrastruktur internet.

Dari sisi keamanan, CARAKA dirancang untuk memenuhi beberapa kebutuhan utama, yaitu kerahasiaan data (*confidentiality*), integritas dan autentikasi pesan (*integrity and authentication*), perlindungan terhadap *replay attack*, privasi *metadata routing*, resistensi terhadap *network flooding* dan *Sybil attack*, serta *forward secrecy* berbasis sesi komunikasi. Seluruh pesan dienkripsi menggunakan mekanisme *end-to-end encryption* sehingga *node* perantara hanya berfungsi sebagai relay tanpa memiliki akses terhadap isi pesan. Selain itu, autentikasi dilakukan pada dua lapisan, yaitu autentikasi per-*hop* untuk *relay* dan autentikasi *end-to-end* pada penerima akhir.

Pemilihan algoritma kriptografi menjadi aspek penting dalam penelitian ini. National Institute of Standards and Technology (NIST) telah menyelesaikan proses standardisasi *Lightweight Cryptography* dan menetapkan keluarga algoritma Ascon sebagai standar resmi melalui NIST SP 800-232. Dibandingkan kandidat *lightweight cryptography* lainnya seperti TinyJambu, GIFT-COFB, Grain-128AEAD, maupun PRESENT, Ascon menawarkan kombinasi keamanan 128-bit, dukungan AEAD, fungsi hash, serta *extendable output function* dalam satu keluarga algoritma yang sama [9].

Oleh karena itu, CARAKA menggunakan Ascon sebagai primitif kriptografi utama. Ascon-AEAD128 digunakan untuk enkripsi payload, Ascon-Hash256 digunakan untuk *hashing* dan sinkronisasi pesan, Ascon-XOF128 digunakan untuk derivasi kunci, serta Ascon-MAC digunakan untuk autentikasi per-*hop*. Untuk pertukaran kunci digunakan X25519, sedangkan identitas *node* direpresentasikan menggunakan pasangan kunci Ed25519.

Secara arsitektural, protokol CLAMP menggunakan struktur paket yang terdiri atas *routing header*, *hop authentication*, dan *encrypted payload*. Pendekatan ini memungkinkan proses *relay* dilakukan tanpa membuka isi pesan. *Routing* dilakukan menggunakan mekanisme *controlled flooding* dengan pembatasan *time-to-live* (TTL), sedangkan sinkronisasi pesan menggunakan pendekatan *store-and-forward epidemic synchronization* berbasis *fingerprint hash*. Dengan desain tersebut, CARAKA mampu mempertahankan keamanan komunikasi sekaligus mengurangi *overhead* protokol dibandingkan beberapa sistem *mesh* yang telah ada.

Penelitian ini menghasilkan lima luaran utama, yaitu: (1) prototipe aplikasi CARAKA Desktop berbasis Windows dan Linux, (2) spesifikasi formal protokol CLAMP, (3) dataset evaluasi performa dan keamanan, (4) laporan penelitian akademik yang mendokumentasikan keseluruhan hasil penelitian, serta (5) repositori *open-source* yang memungkinkan replikasi dan audit independen. Melalui penelitian ini diharapkan dapat diperoleh solusi komunikasi mesh terdesentralisasi yang aman, efisien, dan sesuai dengan kebutuhan komunikasi modern pada lingkungan yang tidak selalu memiliki akses internet. Bagian ini merupakan bagian dokumen yang tampak dan menjadi pembuka bagi dokumen isi. Bagian awal sekurangnya terdiri dari lembar sampul, lembar judul, daftar isi, dan sebagainya.

BAB II

RUMUSAN MASALAH

II.1 RUMUSAN MASALAH

Berdasarkan latar belakang penelitian yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang protokol komunikasi *mesh* terdesentralisasi berbasis *Lightweight Cryptography* menggunakan algoritma Ascon (NIST SP 800-232) yang mampu menyediakan kerahasiaan, integritas, autentikasi, dan privasi metadata pada jaringan komunikasi *offline*?
2. Bagaimana merancang dan mengimplementasikan mekanisme autentikasi multi-hop berbasis Ascon-MAC pada *node relay* sehingga dapat mengurangi *cryptographic overhead* dibandingkan penggunaan tanda tangan digital Ed25519 yang umum digunakan pada sistem komunikasi mesh saat ini?
3. Seberapa besar efisiensi overhead paket yang dapat dicapai oleh mekanisme autentikasi berbasis Ascon-MAC dibandingkan pendekatan berbasis Ed25519 ditinjau dari ukuran autentikasi per paket dan kapasitas *payload* yang tersedia?
4. Bagaimana performa implementasi kriptografi berbasis Ascon dibandingkan implementasi berbasis AES-256-GCM pada aplikasi komunikasi desktop, ditinjau dari parameter *encryption throughput*, *decryption throughput*, ukuran *overhead* paket, *end-to-end latency*, dan *memory footprint*?
5. Bagaimana merancang mekanisme sinkronisasi *store-and-forward* yang bersifat *privacy-preserving* menggunakan Ascon-Hash256 sehingga proses pertukaran *fingerprint* pesan antar *node* pada jaringan *delay-tolerant* dapat dilakukan tanpa mengungkapkan isi pesan kepada *node* perantara?
6. Bagaimana tingkat efektivitas protokol yang diusulkan dalam mendukung komunikasi *mesh offline* yang aman, efisien, dan terdesentralisasi berdasarkan hasil pengujian fungsionalitas, performa, dan keamanan sistem?

BAB III

DESAIN ARSITEKTUR

III.1 OBJEK PENELITIAN

Objek penelitian pada tugas akhir ini adalah sistem komunikasi desktop berbasis jaringan *Peer-to-Peer* (P2P) *Mesh* yang beroperasi dengan paradigma *offline-first*. Sistem dirancang agar tetap dapat berfungsi tanpa ketergantungan terhadap server

pusat maupun koneksi internet sehingga sesuai untuk lingkungan dengan keterbatasan infrastruktur komunikasi, kondisi darurat, maupun kebutuhan komunikasi yang mengutamakan privasi.

Penelitian ini berada pada irisan tiga bidang ilmu utama, yaitu kriptografi terapan, sistem terdistribusi, dan rekayasa perangkat lunak. Dari aspek kriptografi, penelitian memanfaatkan algoritma *Lightweight Cryptography* (LWC) Ascon yang telah ditetapkan sebagai standar resmi oleh NIST melalui NIST SP 800-232 pada tahun 2025. Dari aspek sistem terdistribusi, penelitian mengimplementasikan mekanisme komunikasi *mesh* berbasis *store-and-forward* serta *epidemic synchronization* tanpa ketergantungan terhadap otoritas pusat. Sementara dari aspek rekayasa perangkat lunak, penelitian berfokus pada implementasi sistem yang aman, efisien, dan dapat digunakan pada lingkungan desktop modern [7][8][9].

Lingkungan target sistem meliputi *platform* Windows dan Linux yang saling terhubung melalui jaringan lokal berbasis Ethernet atau Wi-Fi. Setiap *node* berfungsi sebagai pengirim, penerima, sekaligus *relay* yang meneruskan paket menuju *node* tujuan. Dengan pendekatan tersebut, komunikasi tetap dapat berlangsung meskipun tidak tersedia akses internet maupun layanan *cloud* eksternal.

Subjek utama yang dievaluasi dalam penelitian ini meliputi:

1. Protokol CLAMP (*Compact Lightweight Authenticated Mesh Protocol*).
2. Implementasi Ascon-AEAD128 dibandingkan AES-256-GCM.
3. Mekanisme autentikasi multi-*hop* berbasis Ascon-MAC.
4. Mekanisme sinkronisasi pesan berbasis *Epidemic Sync* menggunakan Ascon-Hash256.

Untuk menjaga fokus penelitian, ruang lingkup dibatasi pada implementasi desktop berbasis jaringan LAN/Wi-Fi dengan jumlah node antara 2 hingga 15 perangkat. Penelitian tidak membahas *Post-Quantum Cryptography*, *blockchain*, kecerdasan buatan, maupun media komunikasi radio seperti LoRa dan Bluetooth

III.2 ANALISIS KEBUTUHAN

Analisis kebutuhan dilakukan untuk mengidentifikasi layanan dan karakteristik yang harus dipenuhi oleh sistem CARAKA Desktop

Tabel 3.1 *Tools* dan persyaratan instalasi

NO	TOOLS	PERSYARATAN
1	Rust Compiler & Cargo	Ekosistem Rust Stable Toolchain (Edisi 2021) untuk kompilasi backend aman memori.
2	Tauri Framework	Runtime Tauri v2.0, membutuhkan instalasi Webview2 (Windows) atau WebKitGtk (Linux).

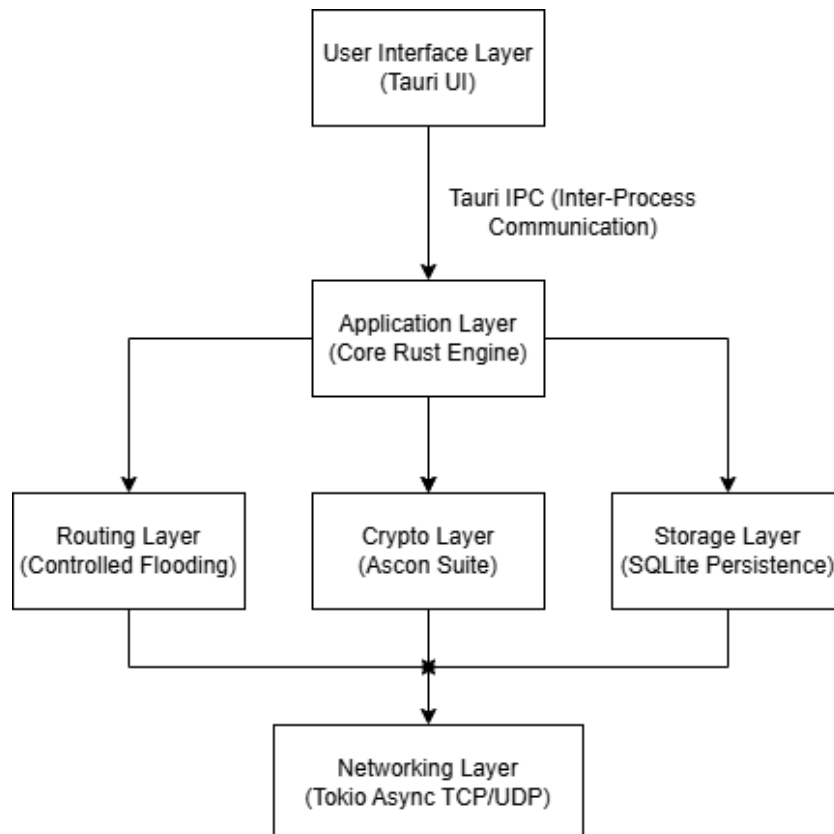
NO	TOOLS	PERSYARATAN
3	SQLite Library	Libsqlite3-sys untuk integrasi engine database relasional lokal berskala ringan.
4	RustCrypto Crate Suite	Dependensi pustaka terverifikasi mencakup ascon-aead, ascon-hash, dan x25519-dalek.
5	Criterion Benchmark	Library pengujian statistik presisi tinggi untuk komparasi <i>micro-benchmark</i> performa <i>cipher</i> .

Berdasarkan persyaratan pada tabel 3.1 digunakan perangkat laptop dengan karakteristik sesuai pada Tabel 3.2.

Tabel 3.2 Karakteristik Perangkat Laptop

Jenis Perangkat	<i>ASUS Vivobook 16x</i>
Sistem Operasi	Windows 11 64-bit
RAM	16 GB
Processor	Intel(R) Core(TM) i5-13420H @2,1 GHz

Gambar 3.1 Arsitektur CARAKA



III.2.1 Kebutuhan Fungsional

Kebutuhan fungsional utama sistem meliputi:

1. Manajemen identitas *node* berbasis pasangan kunci X25519.
2. *Peer discovery* otomatis menggunakan UDP Broadcast.
3. Enkripsi *end-to-end* menggunakan Ascon-AEAD128.
4. Autentikasi multi-hop menggunakan Ascon-MAC.
5. Routing paket menggunakan mekanisme *Controlled Flooding*.
6. Pencegahan *replay attack* menggunakan Packet ID Cache dan *timestamp validation*.
7. Penyimpanan pesan terenkripsi menggunakan mekanisme *Store-and-Forward*.
8. Sinkronisasi pesan menggunakan *Epidemic Sync* berbasis Ascon-Hash256.
9. Antarmuka pengguna grafis untuk pengelolaan identitas dan komunikasi.

III.2.2 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional yang harus dipenuhi meliputi:

Tabel 3.3 Kebutuhan Non-Fungsional Sistem CARAKA Desktop.

Kode	Kategori	Kebutuhan
NF-01	Keamanan (<i>Security</i>)	Seluruh implementasi kriptografi harus menggunakan pustaka yang telah diaudit komunitas (RustCrypto Project) tanpa implementasi primitif kriptografi secara mandiri (<i>no custom crypto</i>). Sistem harus memenuhi properti <i>confidentiality</i> , <i>integrity</i> , <i>authenticity</i> , <i>replay resistance</i> , dan <i>best-effort forward secrecy</i> .
NF-02	Efisiensi (<i>Efficiency</i>)	Total overhead kriptografi per paket tidak boleh melebihi 62 byte. Latensi pengiriman pesan pada jaringan LAN (hop tunggal) tidak boleh melebihi 100 ms dalam kondisi normal. <i>Encryption throughput</i> Ascon-AEAD128 untuk pesan berukuran 256 byte harus dapat diukur dan didokumentasikan.
NF-03	Keandalan (<i>Reliability</i>)	<i>Message Delivery Ratio</i> pada topologi linear 5 hop harus $\geq 95\%$ untuk 1000 pesan yang

Kode	Kategori	Kebutuhan
		dikirim dalam kondisi jaringan normal. Sistem harus beroperasi secara stabil tanpa <i>crash</i> selama minimal 1 jam pengujian berkelanjutan.
NF-04	Portabilitas (<i>Portability</i>)	Aplikasi harus dapat dikompilasi dan dijalankan pada Windows 10/11 dan Ubuntu 22.04 LTS tanpa modifikasi kode yang signifikan.
NF-05	<i>Maintainability</i>	Seluruh modul harus memiliki unit test dengan <i>code coverage</i> minimal 80% pada fungsi kriptografi inti. Kode sumber harus terdokumentasi menggunakan <i>doc-comment</i> standar Rust.
NF-06	Privasi (<i>Privacy</i>)	Sistem tidak boleh mengirimkan data apa pun ke server eksternal atau pihak ketiga. Seluruh data harus tersimpan secara lokal dalam bentuk terenkripsi.

III.2.3 Kebutuhan Perangkat Keras

Tabel 3.4 Spesifikasi Minimum Perangkat Keras

Komponen	Spesifikasi Minimum	Spesifikasi Rekomendasi
Prosesor	Dual-Core 64-bit, 1,6 GHz	Quad-Core 64-bit, 2,0 GHz atau lebih
Memori (RAM)	2 GB	4 GB atau lebih
Penyimpanan	500 MB ruang kosong	2 GB atau lebih
Antarmuka Jaringan	NIC Wi-Fi atau Ethernet	Gigabit Ethernet
Sistem Operasi	Windows 10 (64-bit) atau Ubuntu 22.04 LTS	Windows 11 atau Ubuntu 24.04 LTS

III.2.4 Kebutuhan Perangkat Lunak

Tabel 3.5 *Stack* Teknologi Perangkat Lunak CARAKA Desktop

Komponen	Teknologi	Versi	Justifikasi
Bahasa Implementasi	Rust	1.78+ (stable)	Menyediakan <i>memory safety</i> tanpa <i>garbage collector</i> dan memiliki ekosistem kriptografi yang matang
Pustaka Kriptografi	ascon-aead (RustCrypto)	0.2.x	Implementasi Ascon-AEAD128 yang telah diaudit komunitas
Pertukaran Kunci	x25519-dalek	2.x	Implementasi X25519 yang digunakan secara luas dan telah diaudit
Derivasi Kunci (KDF)	hkdf + sha2	Terbaru	Implementasi standar HKDF-SHA256
Runtime Asinkron	Tokio	1.x	Framework async Rust untuk I/O jaringan
Framework GUI	Tauri	2.x	Framework desktop lintas platform berbasis Rust
Basis Data Lokal	SQLite (rusqlite)	Terbaru	Penyimpanan lokal ringan dan portabel
Benchmarking	Criterion.rs	0.5.x	Framework benchmarking statistik untuk Rust
Serialisasi	bincode / format biner kustom	Terbaru	Efisiensi serialisasi paket protokol
Sistem Build	Cargo + Node.js	Terbaru	Sistem build standar ekosistem Rust dan Tauri

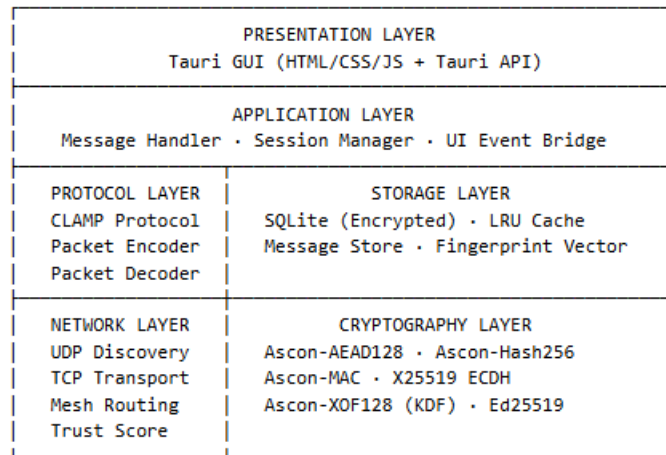
Tabel 3.6 Depedensi Perangkat Pengembangan

Perangkat	Fungsi
cargo-criterion	Menjalankan dan menghasilkan laporan benchmark
rustfmt	Melakukan format kode sesuai standar Rust
clippy	Analisis statis dan pemeriksaan kualitas kode
cargo-tarpaulin	Mengukur <i>code coverage unit test</i>
Wireshark	Analisis dan verifikasi paket jaringan
iperf3	Pengukuran throughput jaringan sebagai <i>baseline</i>

III.3 DESAIN RANCANGAN

III.3.1 Arsitektur Sistem

Gambar 3.2 Arsitektur Keseluruhan Sistem



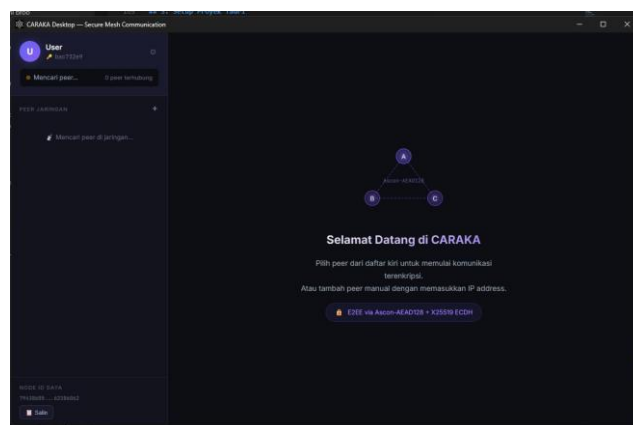
Arsitektur CARAKA Desktop menggunakan pendekatan *layered architecture* yang memisahkan fungsi antarmuka pengguna, logika aplikasi, protokol komunikasi, penyimpanan data, jaringan, dan kriptografi.

Lapisan kriptografi menjadi inti keamanan sistem dengan memanfaatkan:

- Ascon-AEAD128 untuk enkripsi *end-to-end*.
- Ascon-MAC untuk autentikasi *relay*.
- Ascon-Hash256 untuk *fingerprint* pesan.
- Ascon-XOF128 untuk derivasi kunci.
- X25519 sebagai mekanisme pertukaran kunci.
- Ed25519 sebagai identitas *node*.

Pemisahan lapisan tersebut bertujuan meningkatkan modularitas, *maintainability*, dan keamanan sistem.

Gambar 3.3 Antarmuka Caraka



III.3.2 Desain Protokol CLAMP

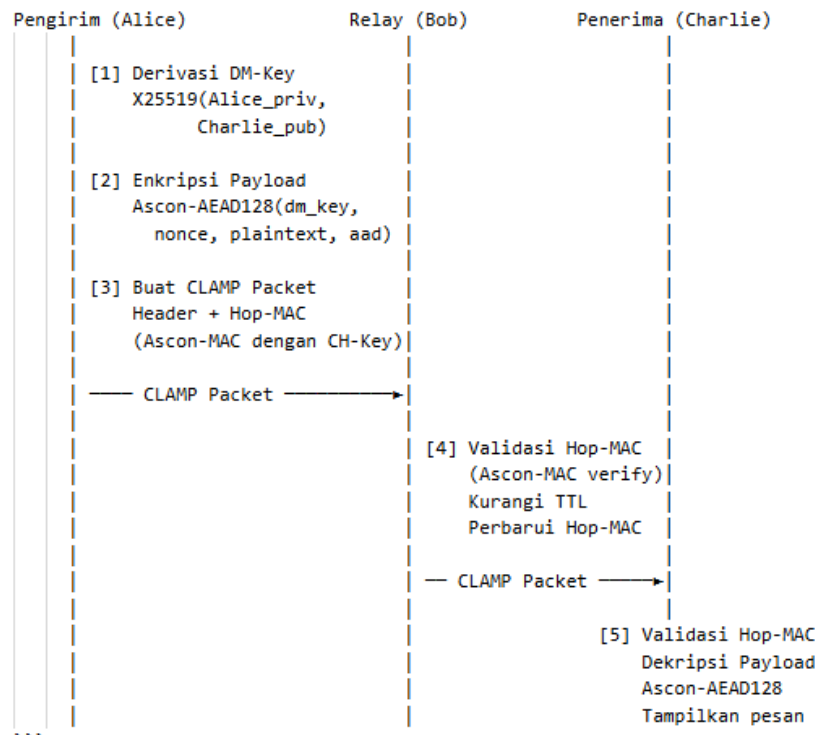
CLAMP (*Compact Lightweight Authenticated Mesh Protocol*) merupakan protokol komunikasi yang diusulkan pada penelitian ini. Protokol dirancang untuk meminimalkan *overhead* kriptografi pada komunikasi multi-hop tanpa mengurangi jaminan keamanan.

Struktur paket CLAMP terdiri atas:

1. *Routing Header*.
2. *Hop Authentication*.
3. *Encrypted Payload*.

Autentikasi *relay* dilakukan menggunakan Ascon-MAC sehingga tidak memerlukan tanda tangan Ed25519 pada setiap paket. Pendekatan ini diperkirakan mampu mengurangi *overhead* autentikasi dari 64-byte menjadi 16-byte atau sekitar 75%.

Gambar 3.4 Skema Alur Kerja Pengiriman Pesan *End-to-End*



III.3.3 Desain Mekanisme Sinkronisasi

Sinkronisasi pesan dilakukan menggunakan pendekatan *Epidemic Sync* berbasis *Message Fingerprint Vector*. Setiap fingerprint dibentuk menggunakan Ascon-Hash256 sehingga *node* dapat mengetahui pesan yang belum dimiliki tanpa mengakses isi pesan. Pendekatan ini mempertahankan *sifat End-to-End Encryption* sekaligus memungkinkan proses *store-and-forward* pada jaringan yang bersifat *delay-tolerant*.

III.3.4 Desain Evaluasi Penelitian

Evaluasi dilakukan pada dua tingkat pengujian.

Microbenchmark Kriptografi

Tabel 3.7 Parameter Evaluasi Microbenchmark Kriptografi.

<i>Parameter</i>	<i>Satuan</i>	<i>Deskripsi</i>
<i>Encryption Throughput</i>	MB/s	Kecepatan proses enkripsi terhadap berbagai ukuran pesan.
<i>Decryption Throughput</i>	MB/s	Kecepatan proses dekripsi terhadap berbagai ukuran pesan.
<i>Waktu Komputasi MAC</i>	μ s	Waktu yang diperlukan untuk menghasilkan dan memverifikasi Ascon-MAC.
<i>Waktu Derivasi Kunci</i>	μ s	Waktu yang dibutuhkan untuk menghasilkan kunci sesi menggunakan mekanisme derivasi kunci.
<i>Memory Footprint</i>	KB	Besarnya penggunaan memori selama operasi kriptografi berlangsung.

Perbandingan dilakukan antara:

- Ascon-AEAD128
- AES-256-GCM
- ChaCha20-Poly1305

Network-Level Benchmark

Tabel 3.8 Parameter Evaluasi Network-Level Benchmark

<i>Parameter</i>	<i>Satuan</i>	<i>Deskripsi</i>
<i>End-to-End Latency</i>	ms	Waktu yang dibutuhkan paket sejak dikirim hingga diterima oleh node tujuan.
<i>Message Delivery Ratio</i>	%	Persentase pesan yang berhasil diterima dibandingkan jumlah pesan yang dikirim.
<i>Ciphertext Expansion</i>	%	Persentase pertambahan ukuran paket setelah proses enkripsi dibandingkan ukuran <i>plaintext</i> .
<i>Hop Overhead</i>	Byte	Besarnya <i>overhead</i> tambahan yang ditambahkan pada setiap <i>relay (hop)</i> selama proses forwarding.
<i>Sync Throughput</i>	Pesan/detik	Jumlah pesan yang dapat disinkronkan per detik pada mekanisme <i>Epidemic Sync</i> .

Pengujian dilakukan pada topologi Linear, Star, dan Mesh Acak untuk mengevaluasi performa protokol pada berbagai kondisi jaringan. Hasil evaluasi ini digunakan untuk menjawab rumusan masalah penelitian terkait efisiensi *overhead*, performa kriptografi, dan efektivitas komunikasi mesh offline berbasis *Lightweight Cryptography*.

DAFTAR PUSTAKA

- [1] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Boston, MA, USA: Pearson, 2011.
- [2] B. Schneier, *Data and Goliath: The Hidden Battles to Collect Your Data and Control Your World*. New York, NY, USA: W. W. Norton & Company, 2015.
- [3] I. F. Akyildiz and X. Wang, *Wireless Mesh Networks*. Hoboken, NJ, USA: John Wiley & Sons, 2009.
- [4] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Harlow, United Kingdom: Pearson, 2023.
- [5] Meshtastic Project, “Encryption Technical Reference,” 2025. [Online]. Available: <https://meshtastic.org/docs/development/reference/encryption-technical/>
- [6] Meshtastic Project, “Known Limitations of Meshtastic Encryption,” 2025. [Online]. Available: <https://meshtastic.org/docs/about/overview/encryption/limitations/>
- [7] Berty Technologies, “Wesh Protocol Documentation,” 2024. [Online]. Available: <https://berty.tech/docs/protocol/>
- [8] Berty Technologies, “Challenges in Building a Distributed Messaging System,” 2024. [Online]. Available: <https://berty.tech/challenges>
- [9] Briar Project, “How Briar Works,” 2024. [Online]. Available: <https://briarproject.org/how-it-works/>
- [10] D. Tarr, C. Lavoie, A. Meyer, and A.-M. Kermarrec, “Secure Scuttlebutt: An Identity-Centric Protocol for Subjective and Decentralized Applications,” in *Proceedings of the ACM Conference on Information-Centric Networking (ICN)*, New York, NY, USA, 2019, pp. 143–153.
- [11] National Institute of Standards and Technology (NIST), “Lightweight Cryptography Project,” 2023. [Online]. Available: <https://csrc.nist.gov/projects/lightweight-cryptography>

- [12] National Institute of Standards and Technology (NIST), *NIST Special Publication 800-232: Ascon-Based Lightweight Cryptography Standards for Constrained Devices*. Gaithersburg, MD, USA: NIST, 2025.
- [13] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  ffer, “Ascon v1.2: Lightweight Authenticated Encryption and Hashing,” *IACR Cryptology ePrint Archive*, Report 2021/1574, 2021.
- [14] S. Banik et al., “NIST IR 8454: Status Report on the Final Round of the NIST Lightweight Cryptography Standardization Process,” National Institute of Standards and Technology, Gaithersburg, MD, USA, 2023.
- [15] S. Banik et al., “Birthday-Bound Slide Attacks on TinyJAMBU’s Keyed Permutations,” in *Advances in Cryptology – ASIACRYPT 2022*, Cham, Switzerland: Springer, 2022.
- [16] S. Banik, S. K. Pandey, T. Peyrin, Y. Sasaki, S. M. Sim, and Y. Todo, “GIFT: A Small PRESENT – Towards Reaching the Limit of Lightweight Encryption,” *IACR Cryptology ePrint Archive*, Report 2017/622, 2017.
- [17] M. Khairallah, “Security of COFB against Chosen Ciphertext Attacks,” *IACR Cryptology ePrint Archive*, Report 2021/648, 2021.
- [18] A. Bogdanov et al., “PRESENT: An Ultra-Lightweight Block Cipher,” in *Proceedings of the Cryptographic Hardware and Embedded Systems (CHES 2007)*, Berlin, Germany: Springer, 2007, pp. 450–466.
- [19] K. Bhargavan and G. Leurent, “On the Practical (In-)Security of 64-bit Block Ciphers: Collision Attacks on HTTP over TLS and OpenVPN (Sweet32),” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, Vienna, Austria, 2016, pp. 456–467.
- [20] RustCrypto Organization, “RustCrypto Project,” 2026. [Online]. Available: <https://github.com/RustCrypto>
- [21] Tauri Programme, “Tauri Documentation,” 2026. [Online]. Available: <https://tauri.app>
- [22] Tokio Project, “Tokio Asynchronous Runtime for Rust,” 2026. [Online]. Available: <https://tokio.rs>
- [23] Dalek Cryptography Project, “x25519-dalek Documentation,” 2026. [Online]. Available: <https://docs.rs/x25519-dalek>
- [24] SQLite Consortium, “SQLite Documentation,” 2026. [Online]. Available: <https://www.sqlite.org/docs.html>